

EXZIBO

Restaurant Management SaaS

Restaurant Profile Data Flow

Analysis Report

Neon + Drizzle Migration — Pre-Implementation Analysis

Prepared: June 29, 2026

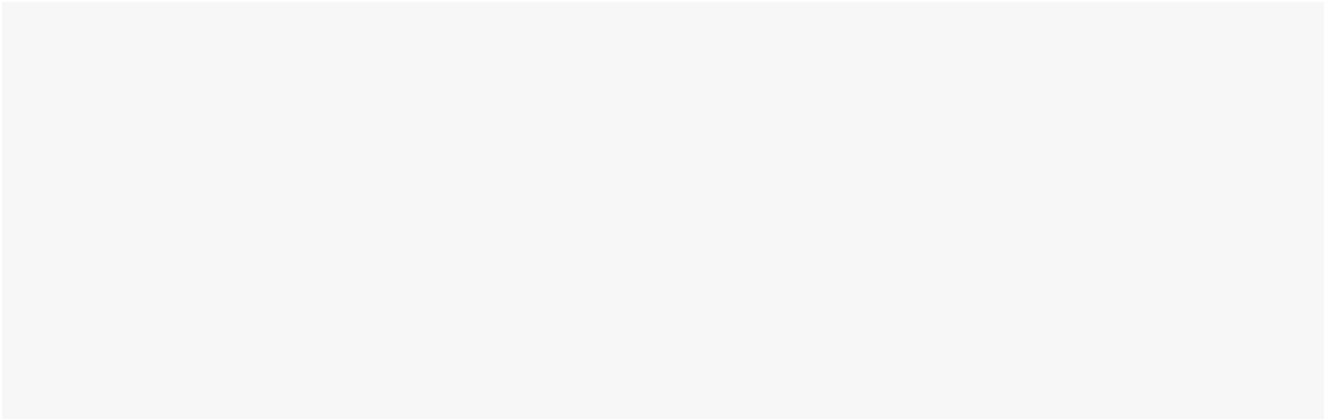
ANALYSIS ONLY — No code changes made

Supabase is untouched • Awaiting implementation approval

```
Frontend (React)
%
% % % Direct Supabase calls (anon/auth key, subject to RLS)
% supabase.from('restaurants').select / insert / update / delete
% supabase.rpc('generate_restaurant_uid')
% supabase.rpc('get_my_restaurant_ids')
%
% % % Server-side API calls (service role key, bypasses RLS)
GET /api/restaurant/:id !' Supabase REST
POST /api/restaurant/update-profile !' Supabase REST
POST /api/restaurant/update-social !' Supabase REST
POST /api/restaurant/upload-logo !' Supabase Storage + REST
```

All restaurant profile reads and writes currently land in Supabase PostgreSQL. No restaurant profile data touches Neon at this stage.

Data Flow Architecture



2. Files That Use Restaurant Profile Data

src/lib/db.js	All CRUD functions — single source of truth for Supabase calls
src/pages/CreateWebsite.jsx	Restaurant creation form — builds the full insert payload
src/pages/AdminDashboard.jsx	Admin panel — reads restaurant row for display, logo, currency
src/pages/ProfilePage.jsx	Displays name, uid, logo (read-only)
src/pages/RestaurantWebsite.jsx	Customer-facing page — reads most profile fields
src/pages/Dashboard.jsx	Lists all restaurants
src/pages/Restaurants.jsx	Public restaurant listing
src/pages/RestaurantListing.jsx	Listing variant
src/pages/DeletedRestaurants.jsx	Soft-deleted restaurant management
src/pages/MasterControl.jsx	Super-admin control panel
src/pages/EditProfile.jsx	Profile editing UI
server.js	4 restaurant API routes (production Express)
vite.config.js	4 restaurant API routes (dev Vite middleware)

3. Supabase Tables Used for Restaurant Profile

restaurants	' Heavily	All profile CRUD operations
restaurant_settings	'L Never	supabase.from("restaurant_settings") not found anywhere in codebase
table_numbers (Supabase)	'L Never (as a table)	Stored as JSONB array inside restaurants.table_numbers column
user_settings	' Yes (adjacent)	Stores global_config (currency, accentColor) — per-user admin prefs, NOT restaurant profile

4. Restaurant Fields Currently Used

Supabase restaurants Table — All Columns

id	UUID PK	All pages and functions
owner_id	UUID NOT NULL !' auth.users	createRestaurant, getRestaurants (via RPC)
name	TEXT NOT NULL	All pages
slug	TEXT UNIQUE	RestaurantWebsite, checkLinkNameTaken, URL routing
uid	TEXT UNIQUE	ProfilePage, AdminDashboard, CreateWebsite
logo	TEXT (full Supabase Storage URL)	AdminDashboard, ProfilePage, RestaurantWebsite
status	TEXT default active	Dashboard, RestaurantWebsite, filtering logic
plan	TEXT default STARTER	RestaurantWebsite, Dashboard plan gating
place	TEXT	RestaurantWebsite header
note	TEXT	Internal dashboard notes
start_date	TIMESTAMPTZ	Plan management (plan start)
end_date	TIMESTAMPTZ	Plan management (plan expiry)
accent_color	TEXT default #6366F1	UI theming
currency	TEXT default INR	AdminDashboard order totals
images	JSONB (array of Supabase URLs)	RestaurantWebsite hero carousel
tables	TEXT	Legacy — not read anywhere in code
table_numbers	JSONB array e.g. [1,2,3]	CreateWebsite (write), RestaurantWebsite (read)
phone	TEXT	RestaurantWebsite contact section
gst	TEXT	Profile / billing display
description	TEXT	RestaurantWebsite about section
chef_info	TEXT	Profile section
servant_info	TEXT	Profile section
social_links	JSONB {}	RestaurantWebsite, update-social API route
rating	TEXT	RestaurantWebsite display
location	TEXT	RestaurantWebsite map/address
additional_info	TEXT	Profile additional details
digital_menu_link	TEXT	CreateWebsite QR menu link
digital_service_bell	BOOLEAN	CreateWebsite service bell toggle
plan_limits	JSONB	Plan feature gating
created_at	TIMESTAMPTZ	Dashboard sorting
is_deleted	BOOLEAN	Soft-delete (added via soft_delete_setup.sql migration)
deleted_at	TIMESTAMPTZ	Soft-delete timestamp
menu_filters	JSONB	Menu filter state persistence
filters_enabled	JSONB	Menu filter enabled/disabled state

5. Existing API Routes

All routes exist in both `server.is` (production) and `viite.config.is` (dev middleware plugin).

GET <code>/api/restaurant/:id</code>	GET	Service-role fetch of single restaurant row — bypasses RLS
POST <code>/api/restaurant/update-profile</code>	POST	Service-role PATCH: only allows name and logo fields
POST <code>/api/restaurant/update-social</code>	POST	Service-role PATCH: <code>social_links</code> object
POST <code>/api/restaurant/upload-logo</code>	POST	Upload WebP to Supabase Storage !' patch logo URL in DB
POST <code>/api/restaurant-db/create</code>	POST	Provision Replit PG isolated schema — separate feature
POST <code>/api/restaurant-db/drop</code>	POST	Drop Replit PG isolated schema — separate feature
GET <code>/api/restaurant-db/list</code>	GET	List Replit PG schemas — separate feature

6. Frontend Field Dependencies by Page

RestaurantWebsite.jsx (Customer-facing)

- slug, uid, name, description, location, phone, rating
- images (carousel array)
- logo (header/branding)
- social_links (footer links)
- table_numbers (table ordering)
- plan, status (feature gating)

AdminDashboard.jsx (Admin panel)

- id, name, uid, logo (header display)
- status, plan (plan indicator)
- currency (via globalConfig from user_settings — NOT from restaurants row)
- logo (live-update via custom DOM event)

CreateWebsite.jsx (Restaurant creation)

- name, slug, uid (auto-generated via Supabase RPC)
- logo (uploaded to Supabase Storage)
- table_numbers (set as JSONB array)
- social_links (Facebook, Instagram, etc.)
- digital_menu_link, digital_service_bell
- place, phone, accent_color, currency, plan, status, images, owner_id

ProfilePage.jsx (Profile display)

- name, uid, logo — read-only display only

7. Neon Schema Field Mapping

id	id	' Match
owner_id (NOT NULL, FK auth.users)	owner_id (nullable)	& p Nullable in Neon — no auth FK yet
name	name	' Match
slug	slug	' Match
uid	uid	' Match
logo (TEXT, full Supabase URL)	logo_key (TEXT, R2 storage key)	'L Name + semantic mismatch
status	status	' Match
plan	plan	' Match
place	place	' Match
note	note	' Match
start_date	start_date	' Match
end_date	end_date	' Match
accent_color	accent_color	' Match
currency	currency	' Match
images (Supabase Storage URLs)	images (designed for R2 keys)	& p Column name matches, semantic differs
phone	phone	' Match
gst	gst	' Match
description	description	' Match
chef_info	chef_info	' Match
servant_info	servant_info	' Match
social_links	social_links	' Match
rating	rating	' Match
location	location	' Match
additional_info	additional_info	' Match
digital_menu_link	digital_menu_link	' Match
digital_service_bell	digital_service_bell	' Match
plan_limits	plan_limits	' Match
is_deleted	is_deleted	' Match
deleted_at	deleted_at	' Match
menu_filters	menu_filters	' Match
filters_enabled	filters_enabled	' Match
created_at	created_at	' Match
(none)	updated_at	' Neon extra — beneficial
tables (TEXT, legacy)	(missing)	& p Legacy field — not read in code, skip
table_numbers (JSONB array in row)	Separate table_numbers table	'L Schema normalization difference

8. Missing Fields / Schema Changes Needed

Neon has logo_key; Supabase has logo	HIGH — every component reads restaurant logo	Add logo TEXT column to Neon restaurants table
table_numbers JSONB vs normalized table	MEDIUM — frontend sends/reads array	Add table_numbers JSONB DEFAULT [] to Neon restaurants table
Migration: add missing columns to Neon restaurants table	LOW — no code reads it	Skip — not needed in Neon
ALTER TABLE restaurants ADD COLUMN IF NOT EXISTS logo TEXT;		
ALTER TABLE restaurants ADD COLUMN IF NOT EXISTS table_numbers JSONB DEFAULT '[]';		
owner_id nullable in Neon	LOW — no auth in Neon yet	Keep nullable; revisit when Better Auth is added
-- These will be generated via: npx drizzle-kit generate && npx drizzle-kit migrate		

Required Neon Migration SQL

9. Recommended Implementation Plan

Analysis only — no code changes have been made. Awaiting approval before any implementation.

Phase A — Neon Schema Patch

- Add logo TEXT and table_numbers JSONB columns to Neon restaurants table
- Update src/db/schema.ts with new columns
- Generate new Drizzle migration file (drizzle-kit generate)
- Apply migration to Neon (drizzle-kit migrate)

Phase B — New Server-Side Neon API Routes

- Add GET /api/neon/restaurant/:id to server.js and vite.config.js
- Add POST /api/neon/restaurant/create to server.js and vite.config.js
- Add PATCH /api/neon/restaurant/:id to server.js and vite.config.js
- Keep ALL existing Supabase routes completely intact and running in parallel

Phase C — Dual-Write (Shadow Write)

- Modify createRestaurant() in src/lib/db.js to write to both Supabase and Neon
- Modify updateRestaurant() in src/lib/db.js to write to both Supabase and Neon
- Supabase remains authoritative — Neon write failure is non-blocking and logged
- No reads switched to Neon yet — all reads still come from Supabase

Phase D — Read Switchover (Later, Separate Approval)

- Once Neon data is validated as consistent with Supabase
- Switch reads for specific routes to Neon (GET /api/neon/restaurant/:id)
- Requires separate review and approval before implementation

10. Risks and Warnings

logo field name mismatch	Ø=Ÿ4 High	Every component reads restaurant.logo — Neon must have logo column before any write
supabase.rpc("get_my_restaurant_ids")	Ø=Ÿ4 High	This RPC has no Neon equivalent — list-fetch must stay on Supabase until auth migrates
supabase.rpc("generate_restaurant_uid")	Ø=ßá Medium	UID generation is a Supabase RPC — need Neon equivalent (sequence or app-level UUID)
Images stay in Supabase Storage	Ø=ßá Medium	Even if row moves to Neon, logo still holds a Supabase Storage URL until R2 is added
owner_id FK not enforced in Neon	Ø=ßá Medium	Neon owner_id has no FK constraint yet — referential integrity not enforced
Dual-write failure window	Ø=ßá Medium	If Supabase write succeeds but Neon fails, DBs diverge — must handle gracefully
table_numbers shape difference	Ø=ßá Medium	Supabase stores [1,2,3] as JSONB in row; code expects this shape — Neon must match
user_settings.global_config stays on Supabase	Ø=ßá Low	globalConfig (currency, accentColor) is per-user admin pref — outside migration scope

11. Exact Files That Will Need Changes

Files That WILL Change (Next Step)

src/db/schema.ts	Add logo TEXT and table_numbers JSONB columns to restaurants table definition
drizzle/migrations/	New migration file generated from schema change via drizzle-kit generate
server.js	Add POST /api/neon/restaurant/create and PATCH /api/neon/restaurant/:id routes
vite.config.js	Add same Neon routes as dev middleware (mirror of server.js additions)
src/lib/db.js	Add dual-write in createRestaurant() and updateRestaurant() — non-blocking shadow write

Files That Must NOT Be Touched

- src/context/AuthContext.jsx — authentication must remain on Supabase
- src/lib/supabase.js — Supabase client configuration untouched
- All Realtime subscription code — stays on Supabase
- All Supabase Storage upload functions — images stay in Supabase Storage
- All menu, order, booking, staff, and notification code — out of scope
- src/lib/env.js, src/lib/previewAuth.js — auth helpers untouched